

Reglib: A Formal Regulatory Library for Lean 4

Design Notes, Directory Structure & Concrete Examples

April 2026

1. Motivation

Mathlib provides Lean 4 with a structured, growing library of formally verified mathematical facts that provers can use as premises. Our regulatory domain requires the same infrastructure: a corpus of formally stated regulatory rules that a compliance checker can reference, compose, and verify against.

Reglib is a Lean 4 library that mirrors Mathlib’s role for regulatory content. Each regulation is a formally typed structure or proposition; each verified compliance check becomes a lemma available as a premise for future proofs. This directly implements the LEGO-Prover pattern (Wang et al., ICLR 2024) and is validated at scale by LeanAgent (Kumarappan et al., ICLR 2025), which proved 155 previously unproven theorems across 23 Lean repositories using exactly this growing-library approach.

The practical integration path with existing tools:

- **LeanDojo** indexes Reglib as a standard Lean 4 project, extracting proof states and tactic traces for retrieval-augmented proving.
- **Lean Copilot’s** SEARCH_PROOF operates over Reglib’s namespace, using compliance lemmas as premises when proving new ones.
- **SELECT_PREMISES** retrieves relevant definitions and rules from Reglib’s embedding matrix when the prover encounters a new compliance goal.

2. Directory Structure

```
Reglib/
lakefile.lean      -- Lean 4 project configuration
Reglib.lean        -- Root import file (like Mathlib.lean)
ICDR/
  2018/
    definitions/
      Core.lean      -- Issuer, Promoter, QIB types
      FinancialTypes.lean -- INR amounts, fiscal years, ratios
      Enums.lean     -- IssueType, SecurityType, etc.
    rules/
      Chapter2_Part1.lean -- Eligibility (Regs 4-8A)
      Chapter2_Part2.lean -- Convertible instruments (Regs 9-13)
      Chapter2_Part3.lean -- Promoter contribution (Regs 14-15)
      Chapter2_Part4.lean -- Lock-in (Regs 16-22)
    schedules/
      ScheduleVI.lean  -- Disclosure requirements
      ScheduleXIX.lean -- Listing conditions
    amendments/
      2020_Amendment.lean -- Gazette notification amendments
      2023_Amendment.lean
```

```

circulars/
  CIR_2016_47.lean    -- Circular clarifications
court_judgements/
  SAT_2021_xxx.lean   -- Tribunal interpretations

```

2.1 Role of Each Directory

Directory	Purpose and Lean Analogy
definitions/	Base types that everything imports. Analogous to Mathlib’s <code>Data/</code> and <code>Order/</code> — the foundational structures.
rules/	Compliance propositions and check functions. Each proved lemma is added to the library. Analogous to Mathlib’s theorem files.
schedules/	Structured data (disclosure templates, fee tables, allotment formats) referenced by rules via <code>as specified in Schedule VI</code> . In Lean: lookup functions and data structures.
amendments/	Superseded versions of rules. Marked with <code>@[deprecated]</code> or a custom <code>@[amended_by]</code> attribute. Enables temporal compliance: “was this IPO compliant under the rules in effect at filing date?”
circulars/	Auxiliary lemmas that refine interpretation. In Lean: type class instances that specialize a general rule. E.g., a circular specifying which accounting standard to use for “net tangible assets.”
court_judgements/	Proved theorems about rule scope. E.g., “Regulation 5(1)(a) does not apply when debarment was lifted before filing” — this is a lemma constraining applicability.

3. Concrete Lean File Examples

3.1 Reglib/ICDR/2018/definitions/Core.lean

This file defines the foundational types that all rules reference.

```

1  -- Reglib/ICDR/2018/definitions/Core.lean
2  -- Core regulatory types for SEBI ICDR 2018
3
4  namespace Reglib.ICDR
5
6  /-- Indian Rupee amount in crore (1 crore = 10 million) -/
7  abbrev INR_Crore := Nat
8
9  /-- Fiscal year identifier (e.g., 2023 for FY 2022-23) -/
10 abbrev FiscalYear := Nat
11
12 /-- Percentage value (0-100) -/
13 abbrev Pct := Nat
14
15 /-- Issue type as defined in Regulation 2 -/
16 inductive IssueType where

```

```

17 | IPO          -- Initial Public Offer (Chapter II)
18 | FPO          -- Further Public Offer (Chapter IV)
19 | RightsIssue  -- Rights Issue (Chapter III)
20 | QIP          -- Qualified Institutions Placement (Chapter VI)
21 | Preferential -- Preferential Issue (Chapter V)
22 | SME_IPO      -- SME IPO (Chapter IX)
23 deriving DecidableEq, Repr
24
25 /-- Security type as defined in Regulation 2(eee) -/
26 inductive SecurityType where
27   | EquityShares
28   | ConvertibleDebt
29   | ConvertiblePreference
30   | Warrants
31   deriving DecidableEq, Repr
32
33 /-- Promoter information -/
34 structure Promoter where
35   is_debarred : Bool
36   is_director_of_debarred_company : Bool
37   is_wilful_defaulter : Bool
38   is_fugitive_economic_offender : Bool
39   debarment_period_over : Bool
40   holding_pct : Pct
41   deriving Repr
42
43 /-- Core issuer structure for ICDR compliance checking -/
44 structure Issuer where
45   -- Regulation 6(1) fields
46   net_tangible_assets_3yr : List INR_Crore
47   operating_profits_3yr : List INR_Crore
48   net_worth_3yr : List INR_Crore
49   monetary_asset_pct : Pct
50   has_committed_excess_monetary : Bool
51
52   -- Regulation 6(2) alternative track
53   issue_size : INR_Crore
54   post_issue_capital : INR_Crore
55
56   -- Regulation 7 conditions
57   has_stock_exchange_approval : Bool
58   has_depository_agreement : Bool
59   has_perm_pan : Bool
60   equity_shares_dematerialised : Bool
61   is_firm_or_llp_converted : Bool
62
63   -- Regulation 5 disqualifications
64   promoters : List Promoter
65   has_outstanding_convertible_securities : Bool
66
67   -- Regulation 8 / 8A
68   ofs_shares_fully_paid : Bool
69   ofs_holding_period_months : Nat
70
71   -- Regulation 14 promoter contribution
72   promoter_post_issue_holding_pct : Pct
73
74   -- Issue metadata

```

```

75   issue_type : IssueType
76   is_entirely_ofs : Bool
77   deriving Repr
78
79 end Reglib.ICDR

```

3.2 Reglib/ICDR/2018/rules/Chapter2_Part1.lean

Eligibility rules (Regulations 4–8A) as Lean propositions.

```

1  -- Reglib/ICDR/2018/rules/Chapter2_Part1.lean
2  -- Chapter II Part I: Eligibility Requirements
3
4  import Reglib.ICDR.definitions.Core
5
6  namespace Reglib.ICDR.Rules
7
8  open Reglib.ICDR
9
10 -- =====
11 -- Regulation 5: Entities NOT eligible for IPO
12 -- =====
13
14 /-- Reg 5(1)(a): No debarred promoters/directors -/
15 def reg_5_1_a (issuer : Issuer) : Prop :=
16   issuer.promoters.all (fun p => !p.is_debarred)
17
18 /-- Reg 5(1)(b): No promoter/director of debarred company -/
19 def reg_5_1_b (issuer : Issuer) : Prop :=
20   issuer.promoters.all (fun p => !p.is_director_of_debarred_company)
21
22 /-- Reg 5(1)(c): No wilful defaulter -/
23 def reg_5_1_c (issuer : Issuer) : Prop :=
24   issuer.promoters.all (fun p => !p.is_wilful_defaulter)
25
26 /-- Reg 5(1)(d): No fugitive economic offender -/
27 def reg_5_1_d (issuer : Issuer) : Prop :=
28   issuer.promoters.all (fun p => !p.is_fugitive_economic_offender)
29
30 /-- Reg 5(1) Explanation: Debarment exception -/
31 def reg_5_1_explanation (issuer : Issuer) : Prop :=
32   issuer.promoters.all (fun p =>
33     !p.is_debarred || p.debarment_period_over)
34
35 /-- Reg 5(2): No outstanding convertible securities -/
36 def reg_5_2 (issuer : Issuer) : Prop :=
37   !issuer.has_outstanding_convertible_securities
38
39 /-- Combined Regulation 5 eligibility gate -/
40 def reg_5_eligible (issuer : Issuer) : Prop :=
41   reg_5_1_a issuer
42   ∧ reg_5_1_b issuer
43   ∧ reg_5_1_c issuer
44   ∧ reg_5_1_d issuer
45   ∧ reg_5_2 issuer
46
47 -- =====

```

```

48 -- Regulation 6(1): Positive eligibility conditions
49 -- =====
50
51 /-- Reg 6(1)(a): Net tangible assets >= 3 crore
52     in each of preceding 3 years -/
53 def reg_6_1_a (issuer : Issuer) : Prop :=
54     issuer.net_tangible_assets_3yr.length = 3
55     ^ issuer.net_tangible_assets_3yr.all (    >= 3)
56
57 /-- Reg 6(1)(a) Proviso: If monetary assets > 50%,
58     must have committed excess to business -/
59 def reg_6_1_a_proviso (issuer : Issuer) : Prop :=
60     issuer.monetary_asset_pct > 50
61     → issuer.has_committed_excess_monetary
62
63 /-- Reg 6(1)(a) Proviso 2: 50% limit does not apply
64     if IPO is entirely OFS -/
65 def reg_6_1_a_proviso2 (issuer : Issuer) : Prop :=
66     issuer.is_entirely_ofs → True
67
68 /-- Reg 6(1)(b): Average operating profit >= 15 crore
69     in preceding 3 years, with profit in each year -/
70 def reg_6_1_b (issuer : Issuer) : Prop :=
71     issuer.operating_profits_3yr.length = 3
72     ^ issuer.operating_profits_3yr.all (    > 0)
73     ^ (issuer.operating_profits_3yr.foldl (    +    ) 0) / 3 >= 15
74
75 /-- Reg 6(1)(c): Net worth >= 1 crore in each of
76     preceding 3 years -/
77 def reg_6_1_c (issuer : Issuer) : Prop :=
78     issuer.net_worth_3yr.length = 3
79     ^ issuer.net_worth_3yr.all (    >= 1)
80
81 /-- Combined Regulation 6(1) with provisos -/
82 def reg_6_1_eligible (issuer : Issuer) : Prop :=
83     reg_6_1_a issuer
84     ^ (reg_6_1_a_proviso issuer ∨ reg_6_1_a_proviso2 issuer)
85     ^ reg_6_1_b issuer
86     ^ reg_6_1_c issuer
87
88 -- =====
89 -- Example: Proving compliance for a concrete issuer
90 -- =====
91
92 /-- A sample compliant issuer for testing -/
93 def sample_issuer : Issuer := {
94     net_tangible_assets_3yr := [10, 12, 15]
95     operating_profits_3yr := [20, 18, 22]
96     net_worth_3yr := [5, 6, 8]
97     monetary_asset_pct := 30
98     has_committed_excess_monetary := true
99     issue_size := 100
100     post_issue_capital := 500
101     has_stock_exchange_approval := true
102     has_depository_agreement := true
103     has_perm_pan := true
104     equity_shares_dematerialised := true
105     is_firm_or_llpConverted := false

```

```

106   promoters := [
107     { is_debarred := false
108       is_director_of_debarred_company := false
109       is_wilful_defaulter := false
110       is_fugitive_economic_offender := false
111       debarment_period_over := false
112       holding_pct := 25 }
113   ]
114   has_outstanding_convertible_securities := false
115   ofs_shares_fully_paid := true
116   ofs_holding_period_months := 18
117   promoter_post_issue_holding_pct := 22
118   issue_type := IssueType.IPO
119   is_entirely_ofs := false
120 }
121
122 /-- Prove sample issuer passes Regulation 5 -/
123 theorem sample_passes_reg5 :
124   reg_5_eligible sample_issuer := by
125     unfold reg_5_eligible reg_5_1_a reg_5_1_b
126     reg_5_1_c reg_5_1_d reg_5_2 sample_issuer
127   simp [List.all]
128
129 /-- Prove sample issuer passes Regulation 6(1)(a) -/
130 theorem sample_passes_reg6_1_a :
131   reg_6_1_a sample_issuer := by
132     unfold reg_6_1_a sample_issuer
133   simp [List.all, List.length]
134
135 end Reglib.ICDR.Rules

```

3.3 Reglib/ICDR/2018/amendments/2020_Amendment.lean

Demonstrating how amendments are tracked in the library.

```

1  -- Reglib/ICDR/2018/amendments/2020_Amendment.lean
2  -- Amendment via Gazette Notification dated Jan 2020
3  -- Modifies Regulation 2(p): expanded definition of
4  -- fugitive economic offender
5
6  import Reglib.ICDR.definitions.Core
7
8  namespace Reglib.ICDR.Amendments
9
10 /-- Original Reg 2(p) definition (2018): fugitive economic
11     offender as declared under Section 12 of FEQ Act 2018 -/
12 @[deprecated "Superseded by 2020 amendment"]
13 def fugitive_offender_2018 (p : Promoter) : Prop :=
14   p.is_fugitive_economic_offender
15
16 /-- Amended Reg 2(p) (2020): includes persons against whom
17     an application has been filed under Section 4 -/
18 def fugitive_offender_2020 (p : Promoter) : Prop :=
19   p.is_fugitive_economic_offender
20   -- Note: the Issuer structure would need an additional
21   -- field to distinguish declared vs application-filed
22

```

```

23 /-- Amendment metadata -/
24 structure AmendmentInfo where
25   gazette_notification : String
26   effective_date       : String
27   regulations_affected : List String
28
29 def amendment_2020_info : AmendmentInfo := {
30   gazette_notification :=
31     "SEBI/LAD-NRO/GN/2020/03"
32   effective_date       := "2020-03-16"
33   regulations_affected := ["2(p)", "5(1)(c)"]
34 }
35
36 end Reglib.ICDR.Amendments

```

3.4 Reglib/ICDR/2018/circulars/CIR_2016_47.lean

Circulars as interpretive lemmas.

```

1  -- Reglib/ICDR/2018/circulars/CIR_2016_47.lean
2  -- SEBI/HO/CFD/DIL/CIR/P/2016/47 dated March 31, 2016
3  -- Clarifies financial statement preparation principles
4
5  import Reglib.ICDR.definitions.Core
6
7  namespace Reglib.ICDR.Circulars
8
9  /-- Circular specifies that consolidated financial
10     statements for Reg 6(1) must follow IndAS and
11     be prepared as per Companies Act 2013 (as amended).
12
13     This constrains HOW net_tangible_assets_3yr,
14     operating_profits_3yr, and net_worth_3yr are
15     computed --- they must use restated, consolidated
16     figures under IndAS. -/
17  axiom cir_2016_47_indas_requirement :
18    ∀ (issuer : Issuer),
19      -- The financial figures in the Issuer structure
20      -- are assumed to be IndAS-restated-consolidated.
21      -- This circular makes that assumption explicit.
22      True
23
24  /-- Circular metadata -/
25  def cir_2016_47_info := {
26    circular_number := "SEBI/HO/CFD/DIL/CIR/P/2016/47"
27    date            := "2016-03-31"
28    subject         := "Financial statement preparation for IPO"
29    regulations_clarified := ["6(1)(a)", "6(1)(b)", "6(1)(c)"]
30  }
31
32  end Reglib.ICDR.Circulars

```

3.5 Reglib.lean — Root Import File

```

1  -- Reglib.lean

```

```

2  -- Root import: provides unified namespace
3
4  import Reglib.ICDR.definitions.Core
5  import Reglib.ICDR.definitions.FinancialTypes
6  import Reglib.ICDR.definitions.Enums
7  import Reglib.ICDR.rules.Chapter2_Part1
8  import Reglib.ICDR.rules.Chapter2_Part2
9  import Reglib.ICDR.rules.Chapter2_Part3
10 import Reglib.ICDR.rules.Chapter2_Part4

```

4. Integration with Lean Copilot and LeanDojo

4.1 LeanDojo Indexing

Once Reglib is a standard Lean 4 project with a `lakefile.lean`, LeanDojo can index it:

1. `lake init Reglib` — initialize the project.
2. Place all `.lean` files in the directory structure.
3. `lake build` — verify everything compiles.
4. LeanDojo extracts proof data (tactic states, premises used) for training or retrieval.

LeanDojo's `ReProver` model can then be fine-tuned on Reglib's proof traces, producing a domain-adapted premise retriever for compliance proofs.

4.2 Lean Copilot Integration

- **SELECT_PREMISES**: Encodes a compliance proof goal into a vector, then searches Reglib's embedding matrix for relevant definitions and lemmas. E.g., when proving a composite eligibility check, it retrieves `reg_5_eligible` and `reg_6_1_eligible` as candidate premises.
- **SEARCH_PROOF**: Uses `ExternalGenerator` pointing to a hosted LLM, with Reglib's lemma library as context. Each newly proved compliance lemma is added to the search space for future proofs.
- **SUGGEST_TACTICS**: When a proof skeleton gets stuck on a subgoal, suggests domain-relevant tactics. For compliance proofs, `omega`, `decide`, and `simp` close most numeric and boolean goals.

4.3 Curriculum Learning (LeanAgent Pattern)

The order in which rules are proved matters. Reglib enables a natural curriculum:

1. **Boolean flags**: `reg_5_1_a` (`is_debarred`), `reg_5_1_d` (`is_fugitive`) — trivial by `decide`.
2. **Numeric thresholds**: `reg_6_1_a` ($\text{NTA} \geq 3 \text{ crore}$) — closed by `omega` or `simp`.
3. **List properties**: `reg_6_1_b` (operating profit in each of 3 years) — requires `List.all` reasoning.
4. **Composite rules**: `reg_5_eligible`, `reg_6_1_eligible` — combine previously proved sublemmas.

5. **Proviso interaction:** `reg_6_1_a_proviso` modifying `reg_6_1_a` — requires implication reasoning.

Each level uses proved lemmas from the previous level as premises, exactly as in LEGO-Prover and LeanAgent.

5. Pipeline Connection

Reglib connects the rule extraction pipeline to the theorem prover:

Pipeline Stage	Reglib Artifact
Rule extraction (<code>maps_to</code>)	→ <code>definitions/Core.lean</code> fields
Metadata enrichment (<code>chapter, part</code>)	→ Directory structure
Schema reconciliation	→ Field types in <code>Issuer</code> structure
Lean code generation	→ <code>rules/Chapter*_Part*.lean</code>
Formal verification	→ Proved theorems added to library
Amendment tracking	→ <code>amendments/*.lean</code>
Cross-reference graph	→ Import dependencies

6. Multi-Jurisdiction Extension

The directory structure naturally extends to other regulatory frameworks:

```
Reglib/
  ICDR/2018/      -- SEBI ICDR (India)
  LODR/2015/      -- SEBI LODR (India, listed entities)
  RegS/           -- SEC Regulation S (US)
  FCA_Listing/    -- FCA Listing Rules (UK)
```

Each jurisdiction has its own `definitions/`, `rules/`, and `amendments/`. Cross-jurisdiction compliance (e.g., a dual-listed issuer) would import from multiple sub-libraries. The shared `Reglib.lean` root provides a unified namespace for multi-jurisdiction queries.